Determinism in LLM Inference and RL Training

Thinking Machines · LLM-42 · bitwise-consistent RL



Re Arch

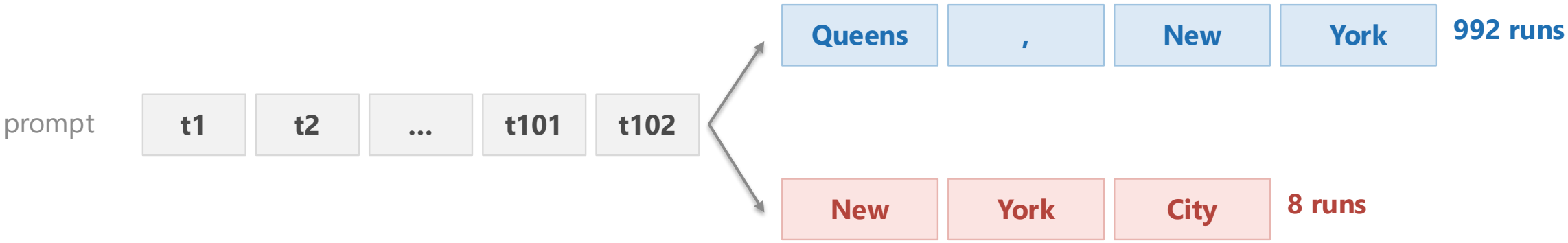
- Oct 2026
- Weiming Hu

Contents

- The symptom, and where the bits actually come from
- Who needs determinism — serving, and RL
- **LLM-42**: determinism as a per-request property
- Across engines: making the trainer reproduce the sampler
- Opportunity: heterogeneous and low-bit

Same prompt, different answers

Qwen3-235B, 1000 samples at temperature 0



Schematic. Prompt: "Tell me about Richard Feynman". 80 unique completions; the most common occurs 78 times.

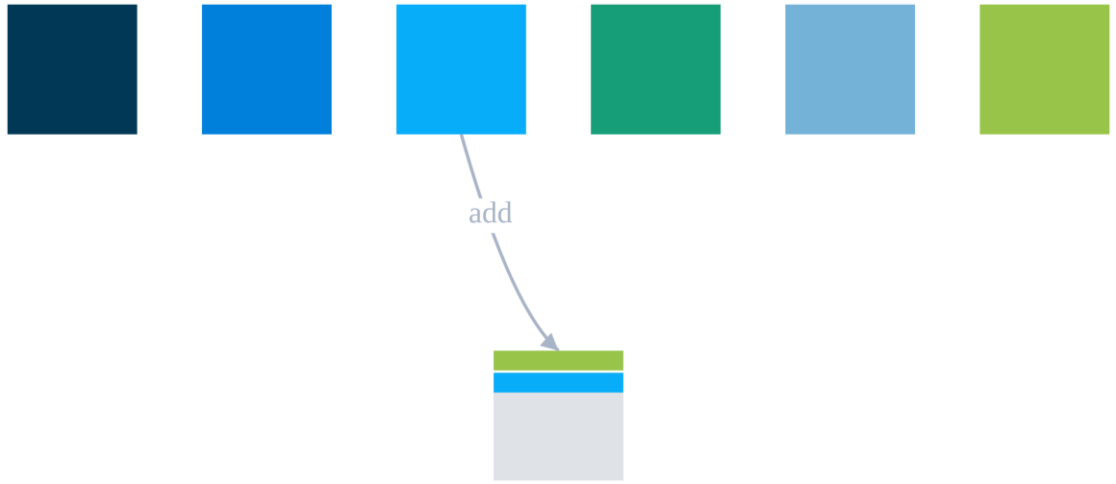
- Same weights, same GPU, greedy decoding — not reproducible
- Once one token differs, autoregression never comes back

→ **Why? Next two slides**

Where the bits come from

The order of accumulation

- `atomicAdd`: the hardware guarantees every add happens, not when
  - the order is whichever core finishes first
- But an LLM forward pass has essentially **no** atomic adds
  - enough parallelism along the batch dim; split reductions with a deterministic clean-up
- So one forward pass **is** run-to-run deterministic
  - then why the previous slide?



Every core's contribution lands — the order does not

→ **Floating-point addition is not associative — order changes the result**

Batch invariance

Deterministic, but not independent of the batch

```
B, D = 2048, 4096
a = torch.linspace(-1000, 1000, B*D).view(B, D)
b = torch.linspace(-1000, 1000, D*D).view(D, D)

out1 = torch.mm(a[:1], b)     # batch of 1
out2 = torch.mm(a, b)[:1]     # batch of 2048

(out1 - out2).abs().max()  ->  1669.25
```

From the server's side

deterministic — same inputs, same output

From your side

the other users decide your batch size, and nobody told you who they were

- Run it twice — same answer. Change the batch — different answer.
- Shape decides the reduction strategy; the reduction order decides the bits

→ **Non-batch-invariant kernel + unpredictable load**

He et al., Defeating Nondeterminism in LLM Inference, Thinking Machines Lab, 2025

Who needs determinism

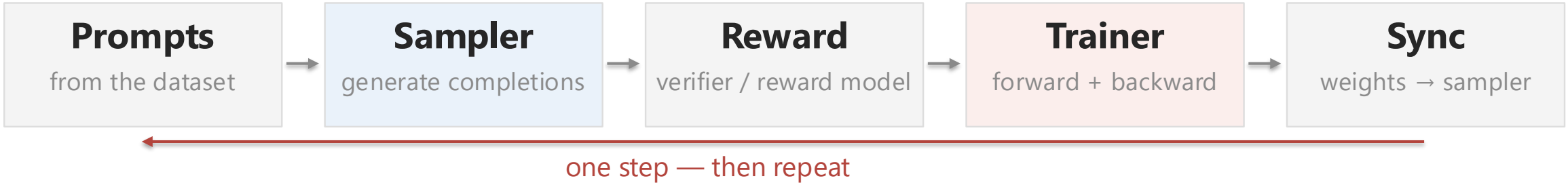
Debugging	a serving bug that only appears at a certain batch composition cannot be re-run
Evaluation	benchmark scores move between runs; rankings between methods can flip
Audit & CI	regression tests and traceability need a run to be repeatable
RL training	the trainer must reproduce the numbers the sampler produced

→ **Three serving scenarios, one RL scenario — same root cause**

He et al., Defeating Nondeterminism in LLM Inference, Thinking Machines Lab, 2025; Gond et al., LLM-42, SOSP 2026 (arXiv:2601.17768)

RL post-training in one slide

Two engines, one policy



Measured on a GRPO run: rollout **66–72%** of step time, reward 5–13%, training 21–23%

rollout
one pass of the sampler: the completions it generated, plus the logprob it assigned to each token

sampler π_rollout
vLLM / SGLang holding the current weights

trainer π_train
re-runs the forward on those same tokens to get its own logprob, then backward + optimizer step

on-policy
the data was produced by the policy being updated: r = π_train(a) / π_rollout(a) = 1

Rollout share: Gao et al., RollPacker (HKUST & Alibaba), arXiv:2509.21009, Table 1

The mismatch problem

Same θ, two implementations

What REINFORCE assumes

$$\theta \leftarrow \theta + \mu \cdot \mathbb{E}_{\underbrace{a \sim \pi(\theta)}_{rollout}}[R(a) \cdot \underbrace{\nabla_\theta \log \pi(a, \theta)}_{training}].$$

What the framework actually runs

$$\theta \leftarrow \theta + \mu \cdot \mathbb{E}_{a \sim \pi_{\text{sampler}}(\theta)}[R(a) \cdot \nabla_\theta \log \pi_{\text{learner}}(a, \theta)].$$

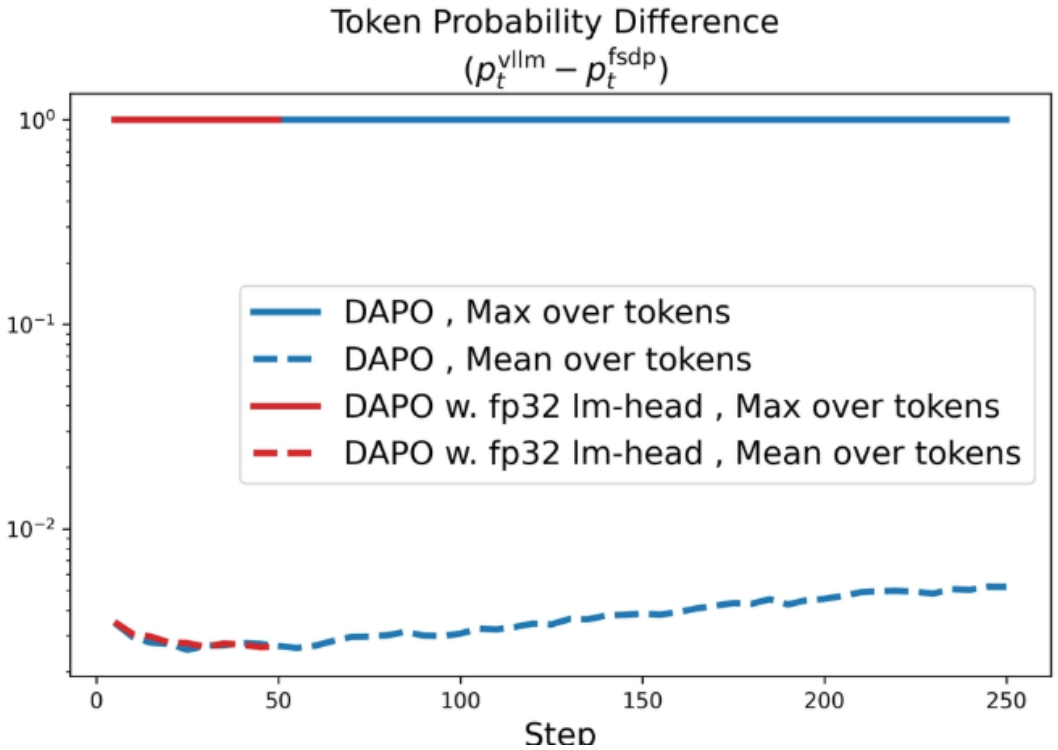
- Sampled from the **inference** engine
  - vLLM / SGLang
- Gradient from the **training** backend
  - FSDP / Megatron
- Same parameters θ — different kernels, different numbers

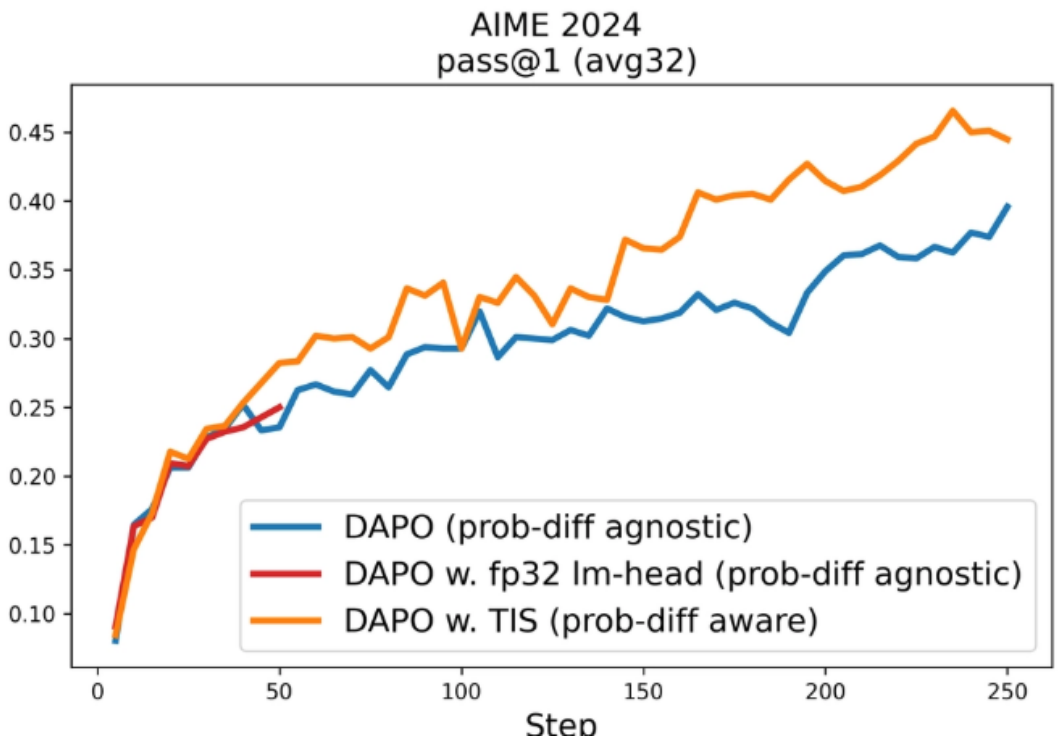
→ **Broken by the implementation gap, not by the algorithm**

Yao, Liu et al. Your Efficient RL Framework Secretly Brings You Off-Policy RL Training, 2025

It is real — and closing it pays

Qwen2.5-32B, 4 nodes × 8 H100



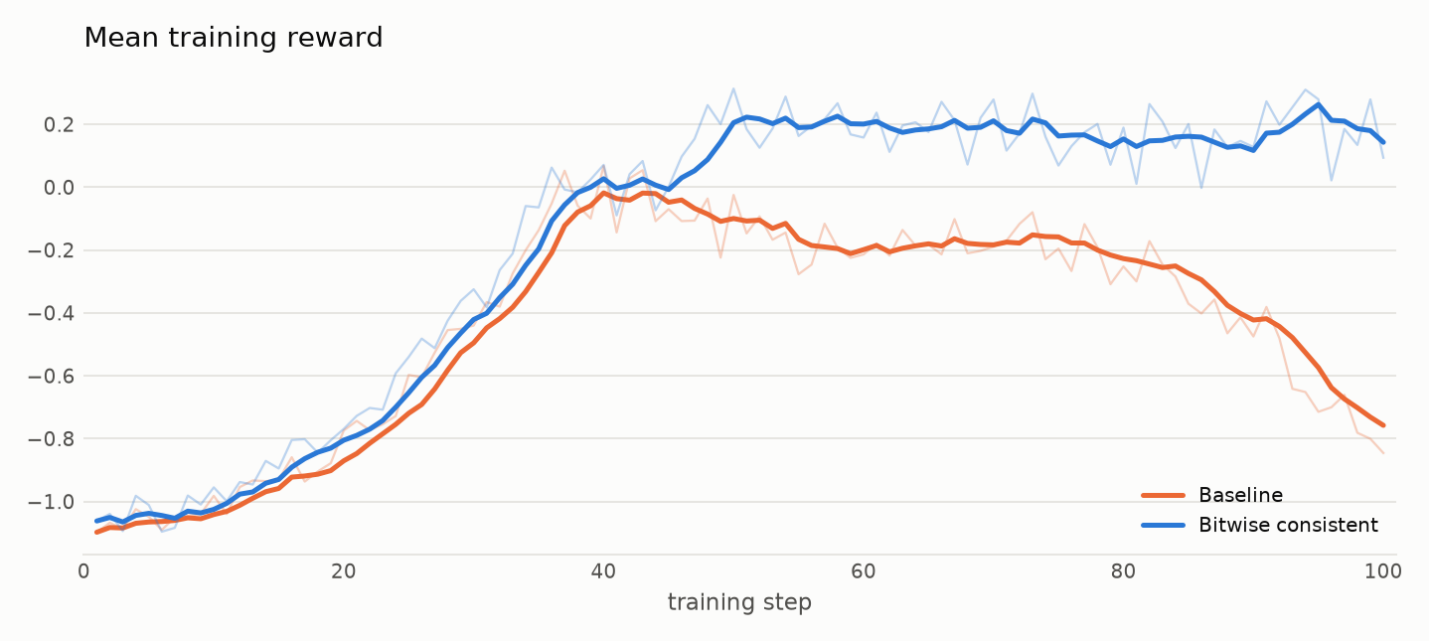


Left: per-token probability difference between the two engines. Right: AIME 2024 pass@1 — TIS is the run that corrects for the difference.

Yao, Liu et al. Your Efficient RL Framework Secretly Brings You Off-Policy RL Training, 2025

And it compounds

DeepSeek-V4-Flash, 100 training steps



- Both runs climb together for ~40 steps
- Then the baseline falls back; the consistent run holds
- Its diff is **0 at every step**

Orange: high-performance baseline. Blue: bitwise-consistent run.

→ **Small at first, then it eats the reward**

Bai & Shen. Finish RL Training Faster, vLLM blog, 2026-10-14

Where to lock it

Determinism has scopes — RL is one scenario on top of them

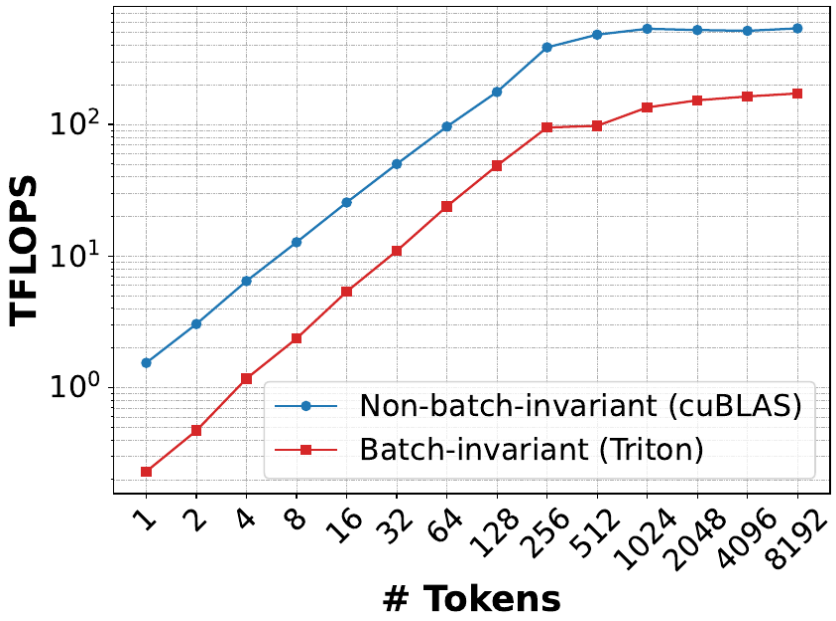
Kernel	same op, same input → same bits	Thinking Machines
Engine	independent of its batch-mates and of when the collectives land	LLM-42
Across engines	sampler and trainer agree on the bits	vLLM × TorchTitan
Across hardware	another GPU or precision	**open problem**

→ **For each work: which scope does it lock — and which one not?**

Constrain: batch-invariant kernels

Thinking Machines

- Three ops are enough: RMSNorm, matmul, attention
  - attention: fix the split **size**, not the split **count**
- Cost: **26 s → 55 s**, 42 s with a better attention kernel
  - Qwen3-8B, 1000 sequences, one GPU
- RL on Bigmath: no correction → reward collapses
  - importance weighting → KL ≈ 0.001; bitwise identical → **KL flat 0**



Batch-invariant Triton GEMM vs cuBLAS

→ **Correct, but every request pays for it**

He et al., Defeating Nondeterminism in LLM Inference, Thinking Machines Lab, 2025; figure: LLM-42, Fig. 4a

LLM-42

Enabling Determinism in LLM Inference with Verified Speculation

Raja Gond[1] · Aditya K Kamath[2] · Arkaprava Basu[3] · Ramachandran Ramjee[1] · Ashish Panwar[1]

[1] Microsoft Research [2] University of Washington [3] Indian Institute of Science

SOSP 2026

arXiv:2601.17768

github.com/microsoft/llm-42

The idea in one line

keep the fast, non-deterministic decode path; prove it right with a fixed-shape verifier, and roll back when it is not

→ **Built on SGLang; you pay only for the traffic that asks for it**

The problem LLM-42 attacks

One deterministic request slows the whole batch

- 11 non-deterministic requests: **931** tok/s
- 11 requests, **one** of them deterministic: **415** tok/s
  - −56%, because the switch is engine-wide
- LLM-42 on the same workload: **911** tok/s

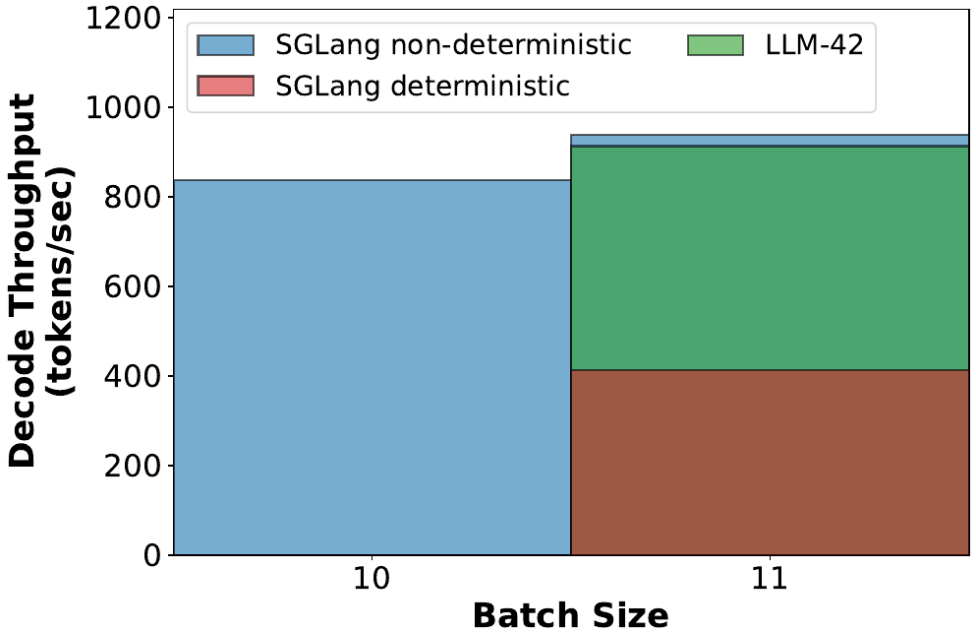


Figure 5: three overlapping bars, not a stack — 931 non-det, 911 LLM-42, 415 SGLang-Det

→ **Determinism should be per request, not per engine**

O1 A consistent prefix stays consistent

ShareGPT · Llama-3.1-8B · batch size 1 as the reference run

- Hundreds of tokens agree, then one diverges
- After that, the two runs essentially never re-converge
  - green ≈ 0 for almost every request
- So: if nothing has diverged yet, the next token probably will not either

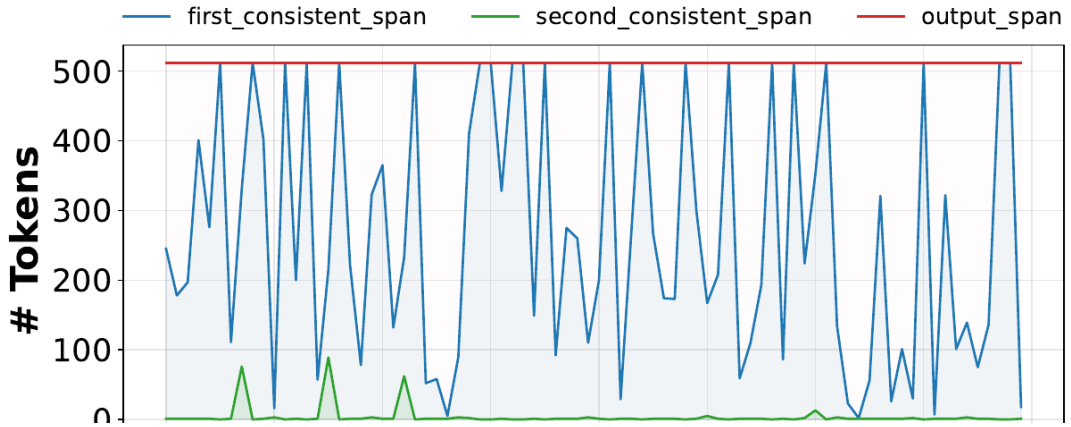


Figure 6: per request — blue = tokens before the first divergence, green = the next agreeing run

→ The fast path is usually already right — it just needs checking

O2 Position invariance

Stronger than determinism, and that is the point

- For a fixed total batch size, an element's output does not depend on **where** it sits in the batch
- cuBLAS GEMM: not batch-invariant, but position-invariant



Figure 7: T1′ == T1″ if the kernel is position-invariant

Category	Operator	Invariant	
		Batch	Position
Matmul	CuBLAS GEMM	✗	✓
Attention	FlashAttention-3‡	✓	✓
Communication	Multimem-based AllReduce*	✓	✓
	Ring-based AllReduce	✗	✗
	Tree-based AllReduce⋆	✓	✓
Normalization	RMSNorm†	✗	✓
	Fused RMSNorm + Residual†	✗	✓

Table 2: invariance properties by operator

→ **This is why a fixed-shape replay can serve as the reference**

Decode – verify – rollback

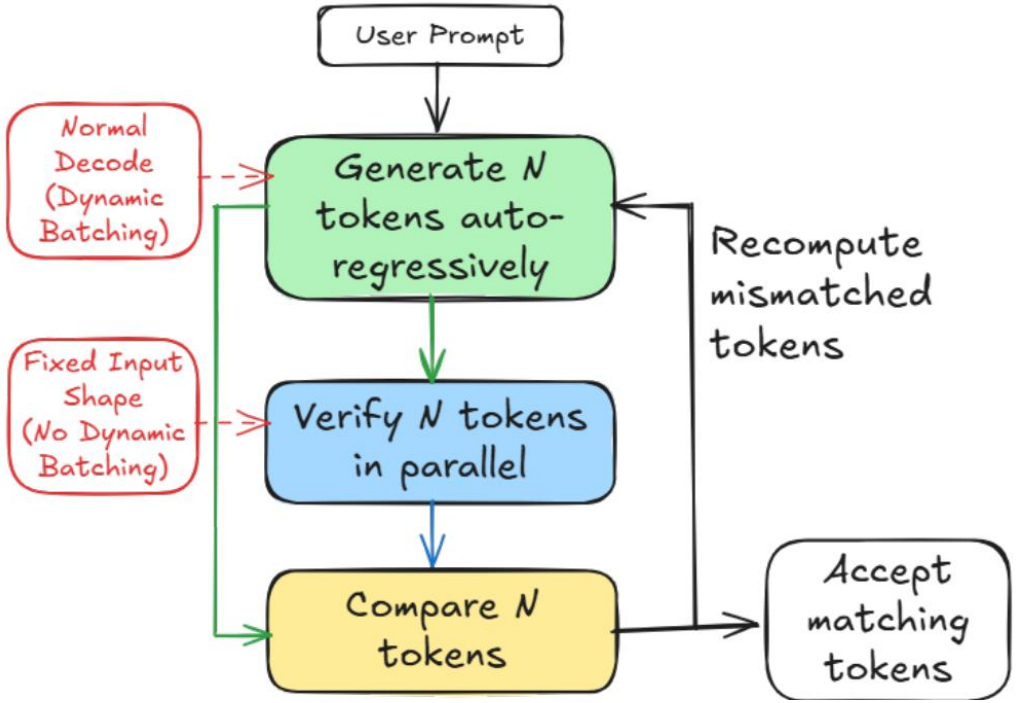
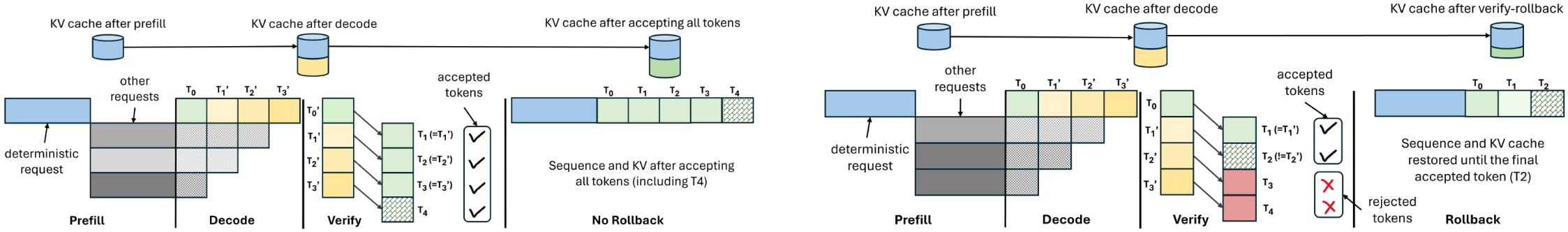


Figure 1: overview of LLM-42

- Fast path decodes with dynamic batching
- Verifier replays the window at a **fixed shape**
  - pads with dummy tokens when the window is short
- Match → accept all, plus one free token
- Mismatch → commit the longest prefix, truncate KV

→ **Determinism moves out of the kernel and into the schedule**

Walking through one window



(a) DVR without rollbacks.

(b) DVR with rollbacks.

Figure 8: (a) all tokens accepted (b) a mismatch at T3 — roll back, then continue

- Decode-time KV entries are overwritten by the verifier's
- Every pass yields ≥ 1 new consistent token — forward progress guaranteed

Not speculative decoding

Speculative Decoding	LLM-42
Fast path generates tokens using some form of approximation	No approximation; only floating-point rounding errors
Low acceptance rate and hence limited speculation depth (2-8 tokens)	High acceptance rate and hence longer speculation (32-64 tokens)
Typically uses separate draft and target models	Uses the same model for decoding and verification

- No draft model
- No approximation on the fast path
  - only floating-point rounding
- That is why acceptance is high

→ **Same model, same prefix — only the reduction order differs**

Gond et al., LLM-42, SOSP 2026 (arXiv:2601.17768)

Cost tracks the deterministic traffic

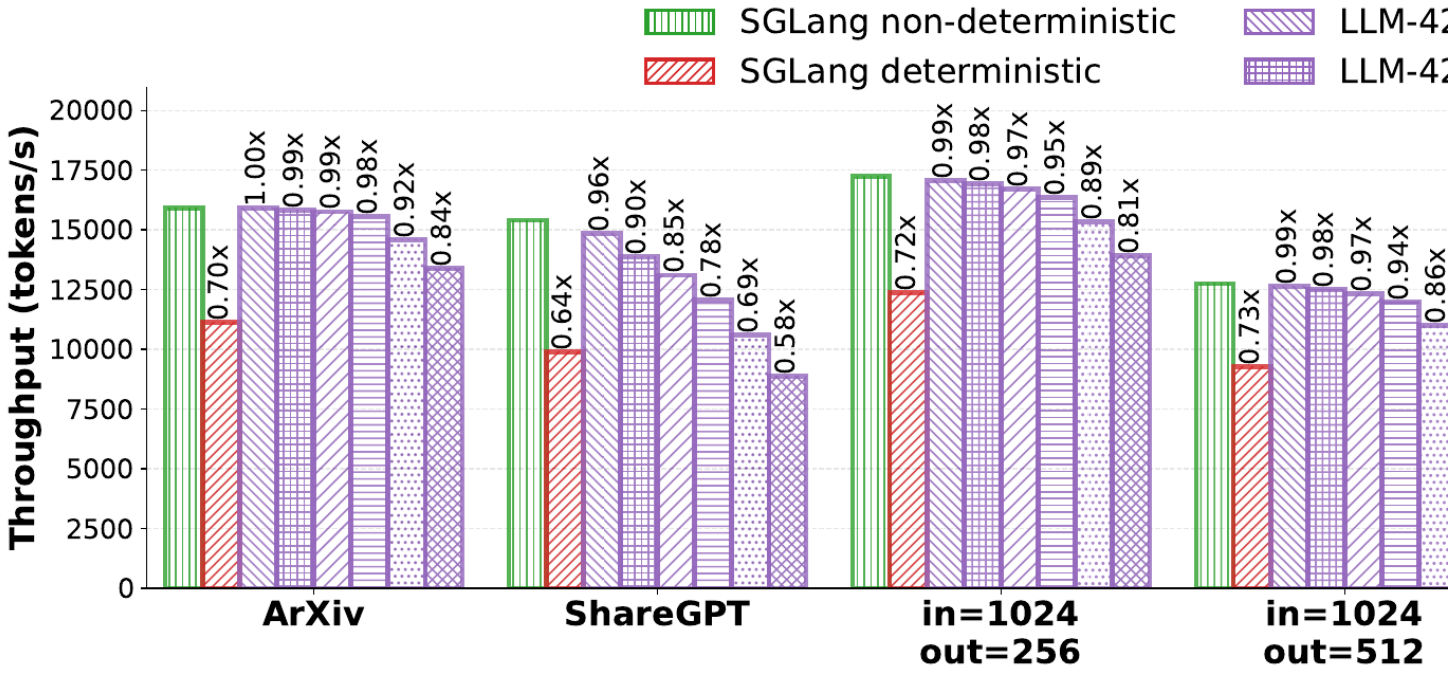


Figure 10: offline throughput; legend = deterministic fraction

- 10–50% deterministic: within **1–2%** of the non-deterministic baseline
- 10%: **33–48%** faster than SGLang-Deterministic
- 100%: 0.81–0.86×

→ **A fixed engine cost becomes a per-request bill**

Rollbacks are rare

Dataset / Config	Deterministic Ratio					
	2%	5%	10%	20%	50%	100%
	Total number of rollbacks					
ArXiv	70	170	372	697	1733	3351
ShareGPT	4	5	10	15	44	96
in=512, out=256	0	0	0	0	0	0
in=1024, out=256	0	1	5	5	9	24
in=1024, out=512	48	106	225	386	792	1536
in=2048, out=256	7	22	79	134	378	724
in=2048, out=512	0	0	0	2	2	4
in=4096, out=512	31	79	98	220	538	1087

- Over half the requests never roll back
- ArXiv @100%: **10.97%** recompute
- ShareGPT @100%: **0.32%**
- Window size is the knob

→ Larger window: verify 0.75 → 0.05 ms, recompute 6.8% → 46.4%

What LLM-42 does not lock

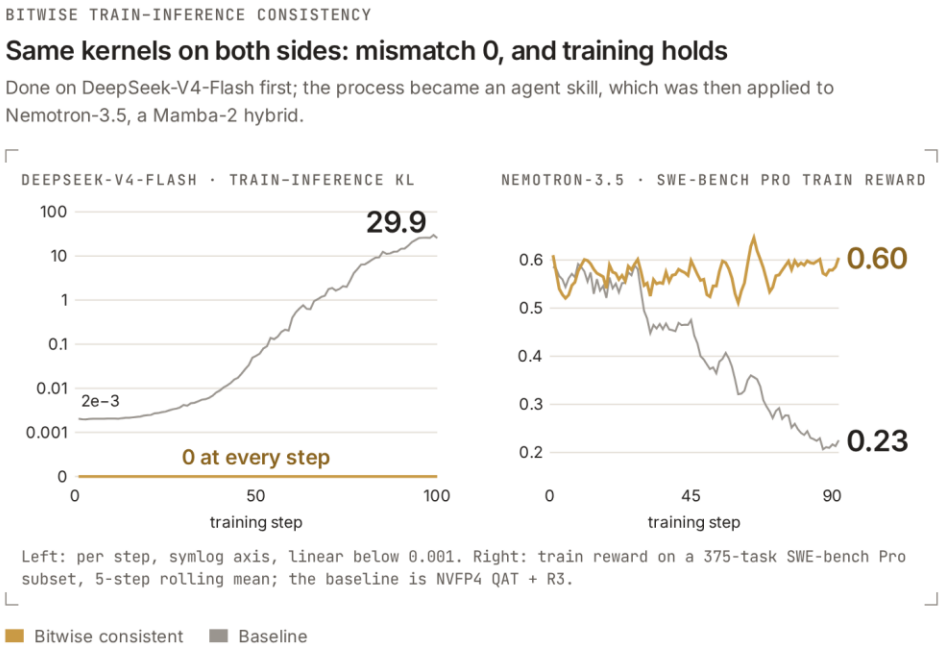
Locked	Open
Kernel — batch-invariant	Across engines — sampler and trainer are two frameworks
Engine — per-request determinism, inside one engine	the verifier replays its own kernels, not the trainer's

- LLM-42 is a serving paper — determinism within one engine
- RL needs the other thing: two engines, one set of weights, the same number

→ **Two engines, not one — and rollout speed is on the line**

Train–inference consistency

vLLM × TorchTitan → the DeepSeek-V4 recipe



Left: DeepSeek-V4-Flash. Right: Nemotron-3.5

What is being claimed

for every sampled token, before each update, rollout and the training forward give the **same** logprob

diff = 0 is a contract, not a tolerance

max, mean, std of the per-token difference and the k3 KL are all exactly 0

same weight version · response mask only · forward only · 1 update per step

→ diff = 0 means one policy — not a good policy

Bai & Shen. Finish RL Training Faster, vLLM blog, 2026-10-14

Across engines: what has been tried

Algorithmic correction	TIS / MIS / Score Centering — leave both engines alone, reweight the update	the gap stays; low precision widens it
Generic invariant kernels	both sides run the same batch-invariant kernels (TML; vLLM × TorchTitan, 2025-11)	bitwise, but **2.4× slower**
QAT	the training forward simulates the deployment quantization	only approximates the deployed numerics
R3	record the rollout's expert routing and replay it in training	removes routing flips only

→ **New in 2026-10: fix the deployment kernels, then call them**

Bai & Shen. Finish RL Training Faster, vLLM blog, 2026-10-14

Three properties, usually two

A Performance

low-precision compute, KV cache, comm and kernel tuning

B Consistency

trainer and rollout compute the same policy

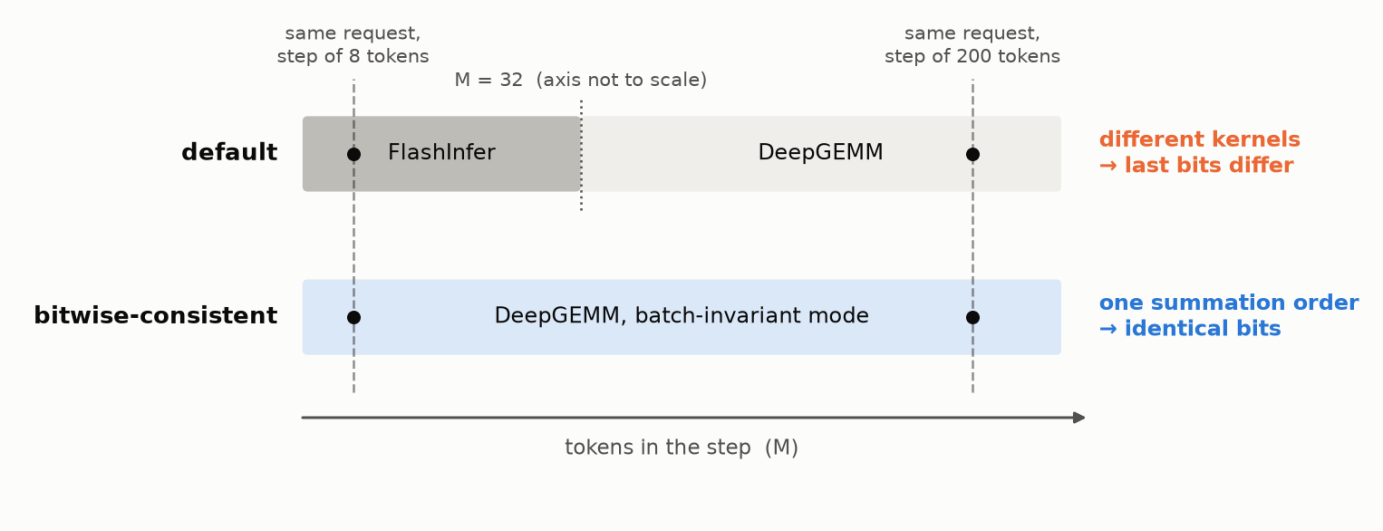
C Low cost

a new model / precision / GPU is cheap to support

- Drop **B** → algorithmic correction; drop **A** → generic kernels; drop **C** → align by hand
- This line keeps A and B, gives up C — and then automates C

→ Give up C, then automate C

Unify: one side defines the numbers



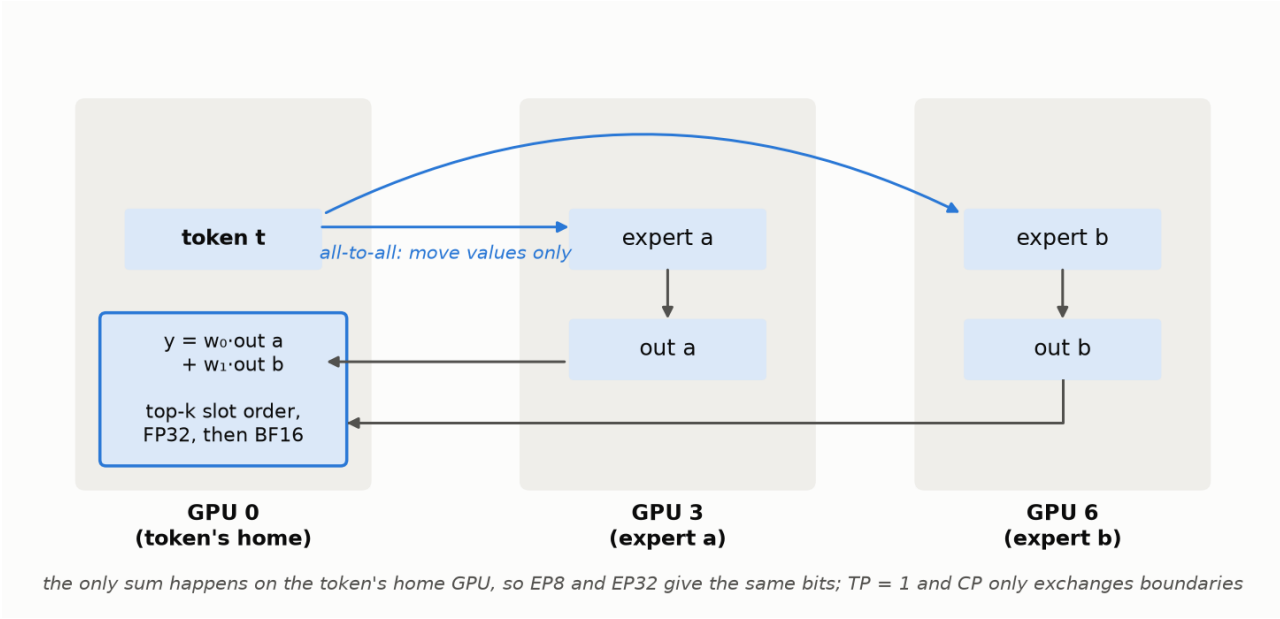
Both are inference kernels: the engine picks FlashInfer below M = 32, DeepGEMM above

- Training forward **calls vLLM's kernels**
- Inference keeps its deployment kernels — modified, not replaced
- Training only adds backward

→ **One implementation, used twice**

What it takes to reach diff = 0

- Kernel determinism
- Batch independence
- Phase consistency (prefill = decode)
- Cross-GPU consistency
- **The same numerical contract on both sides**

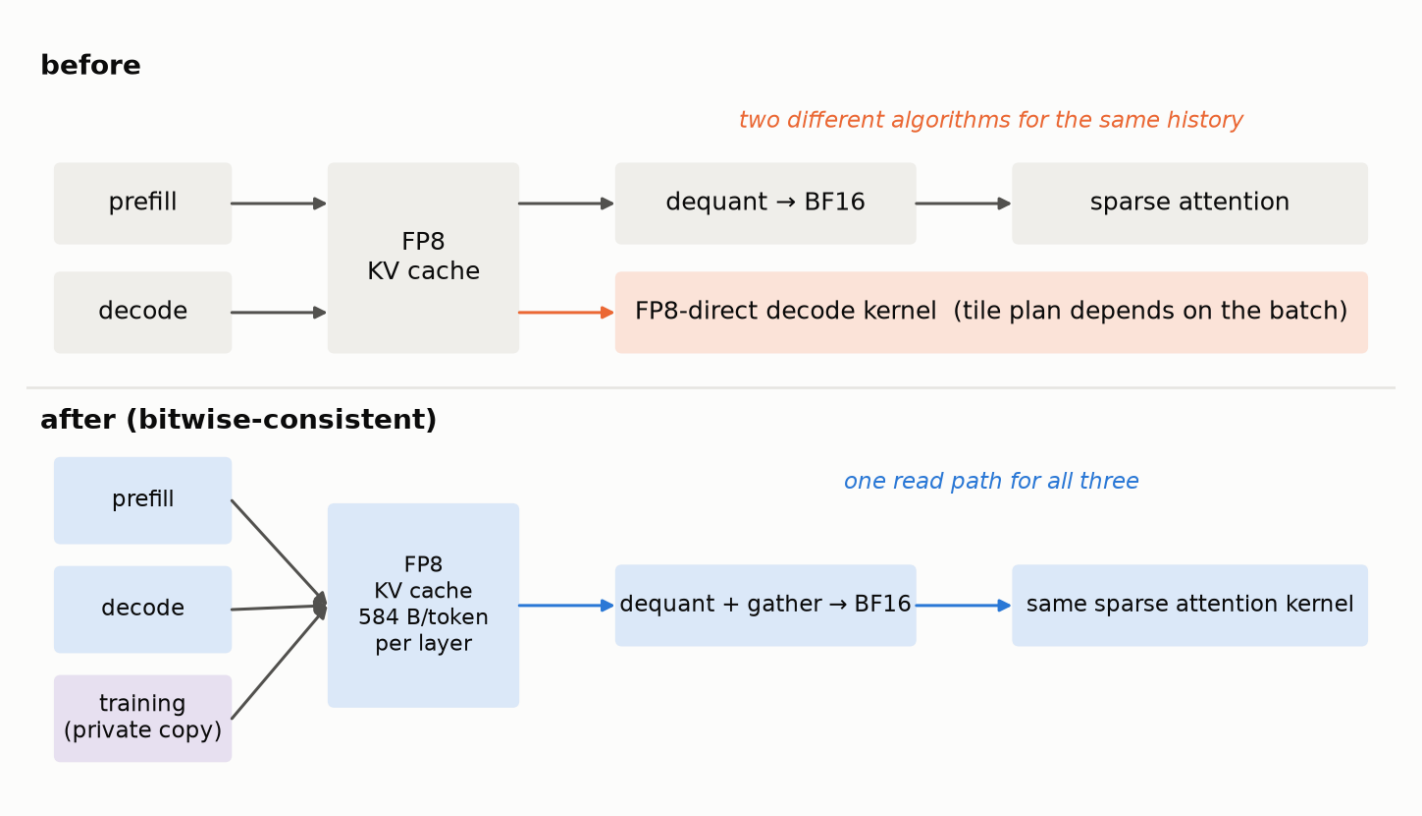


Cross-GPU reduction moved off the compared path

→ **Same format ≠ same contract**

One operator, in detail

FP8 KV cache — what phase consistency costs



Top: before. Bottom: prefill, decode and training share one read path

Kept

FP8 storage — about 57% of BF16, and that sets concurrency

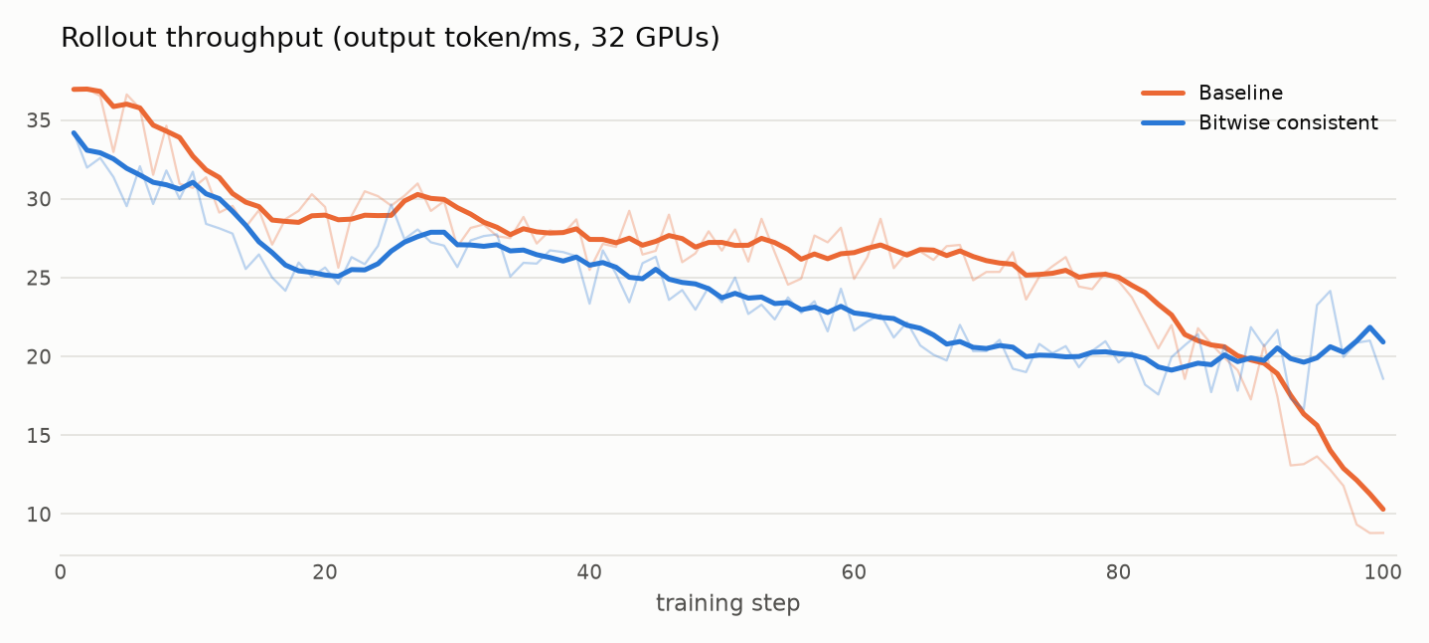
Given up

decode's own FP8-direct kernel, whose work partition followed the batch

The cost

decode now pays a gather and a dequant; training keeps a private FP8 cache so K makes the same round trip

diff = 0 at 91.3% of rollout throughput



- diff = 0 at **every** step
- 24.22 vs 26.53 token/ms over 100 steps
  - the bar they set in advance was 70%
- A year ago the same idea cost **2.4×**

→ **Consistency is no longer the slow option**

Bai & Shen. Finish RL Training Faster, vLLM blog, 2026-10-14

Buying C back

C fails because every new model starts over

Bring diff to 0

fix the weights and the input → find the first operator where the two sides diverge → reduce it to a minimal rerunnable repro → fix → verify in real training

Optimize, with diff still 0

profile the hotspots → A/B one operator at a time → re-verify consistency after every single change

---

What does not count

a speedup that changes the result · a speedup that only shows up in a micro benchmark, not in real rollout · slowing the baseline down to narrow the gap

Cold start: cut **4-6 layers** into a small model that still holds every operator class

→ Mechanically checkable work — what an agent is good at

The 11 gates

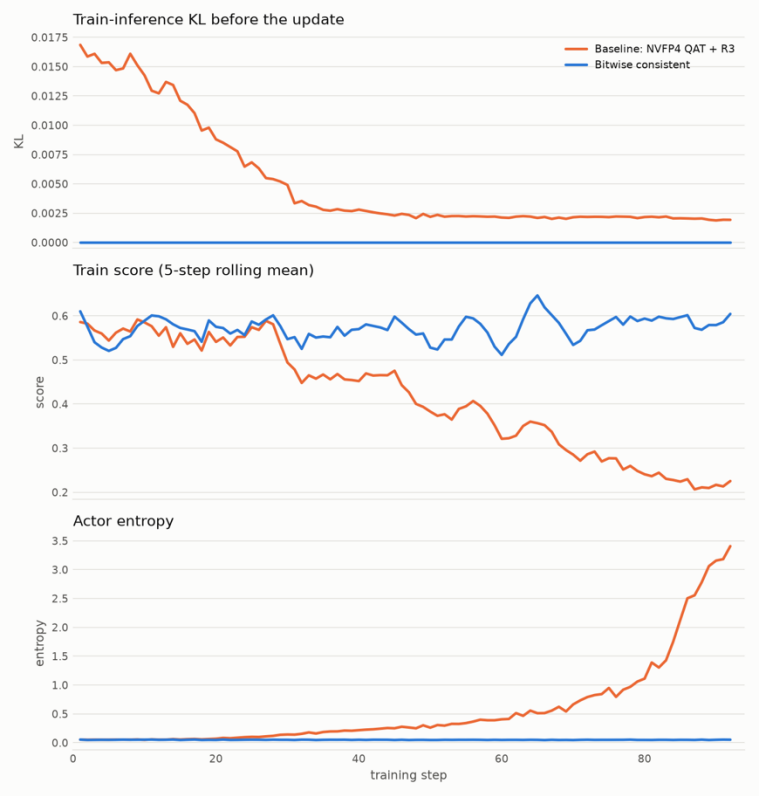
C0–C10, each with an explicit pass condition

C0–C1	**Target and baseline**	list every module with its precision and parallelism; tune the ordinary build, then freeze it
C2–C4	**Inference alone, small model**	a request gives the same result alone and in a batch; whole input matches token by token
C5–C7	**Connect training**	training recomputes what inference produced, matching token by token; gradients checked independently
C8–C9	**Run RL, then full model**	two consecutive updates match on the same weight version; then the full model and a checkpoint restore
C10	**Recover performance**	optimize the costly operators one at a time, re-checking consistency each time

→ **The frozen baseline is the one the agents cannot touch**

Carried over to NVFP4

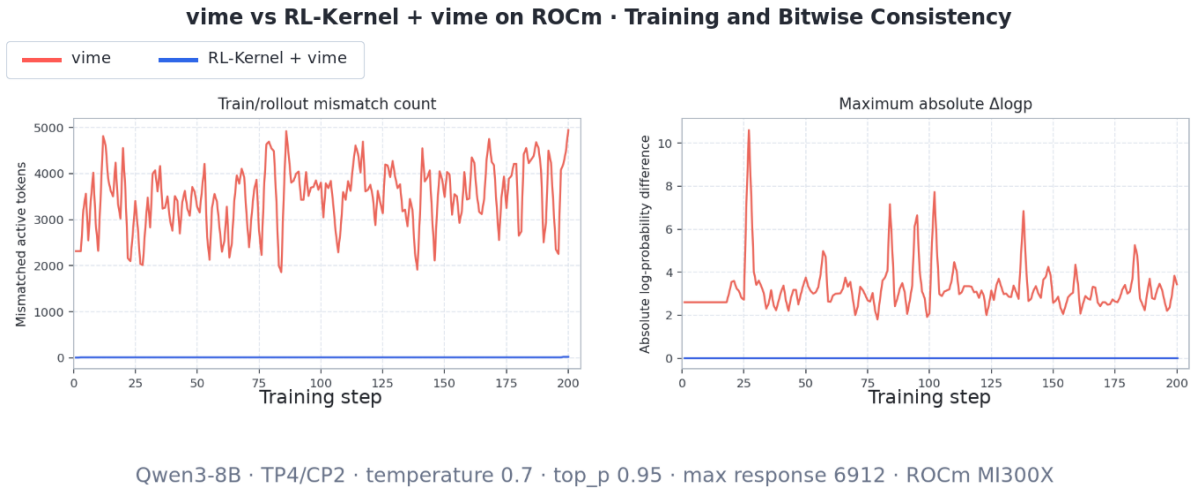
Nemotron-3.5-Lightning-30B-A3B on GB200



- Where the 4 bits are: **routed experts, NVFP4 W4A16**
  - FP8 Mamba projections, FP8 KV cache; BF16 master weights
- What breaks: the expert GEMM
  - one tactic, no split-K, fp32 merge in slot order
- Result: consistent **0.56 → 0.59**, baseline **0.56 → 0.22**
  - baseline entropy 0.05 → 2.6 while its KL falls

→ A falling KL can hide a policy that is coming apart

Alignment is spreading across stacks



Megatron + vLLM on MI300X: 200 GRPO steps, zero mismatch

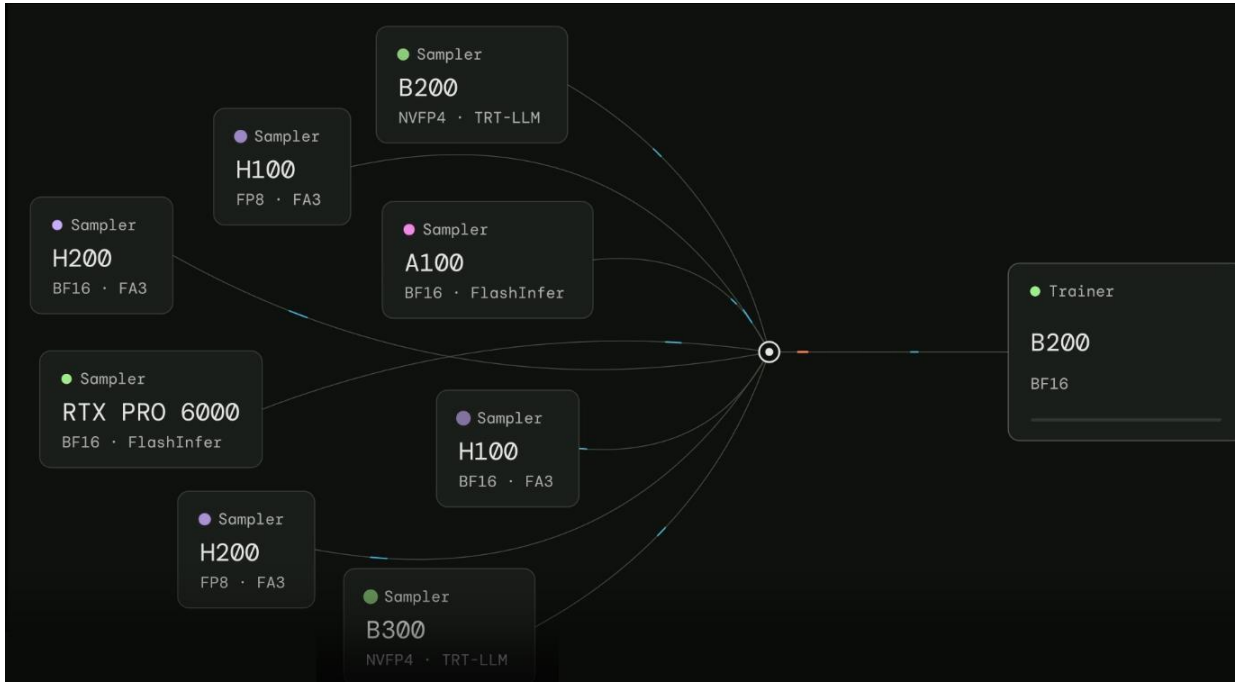
- slime: logprobs aligned on GLM-5
- IsoExec: linear attention
- Miles: NVFP4 / MXFP8 / FP8 recipes, bit-exact quantizer contract

→ Each stack builds its own contract — inside one hardware target

vLLM blog 2026-09-14 (vime × RL-Kernel × AMD); Miles v0.1, LMSYS, 2026-08-18

The other road: tolerate and correct

Modal — mixed-precision, mixed-accelerator samplers



Eight samplers, three precisions, one BF16 trainer

- Six accelerator types, one BF16 trainer
  - NVFP4 on B200 / B300
- Each one is just another distribution q
  - MIS + Score Centering + top-p replay
- Mismatch **4.7×** higher, yet on par
  - sampler cost **~15%** lower

→ **Heterogeneous RL works today — by giving up reproducibility**

Cross-hardware consistency today

ReproFuse Cooper et al. · Georgia Tech, UC Merced · preprint	fixed reduction order → linear layers bitwise identical on Ampere / Ada / Hopper; non-GEMM ops and cross-vendor not covered
Cross-vendor study Fuadi et al. · MBZUAI · preprint	traces the gap to accumulation inside the matrix instruction; measures it, does not fix it
Verde / RepOps Arun et al. · Gensyn AI, NYU, a16z crypto · 2025	replace every operator with a fixed-order one; ~60% overhead on Llama-3.1-1B

→ All of them fall back on one thing: IEEE-754 FMA is portable

arXiv:2609.25624 (ReproFuse); arXiv:2610.05458; Arun et al., Verde, arXiv:2502.19405

Low-bit has no reference semantics

Where I think the opening is

- `scaled.mma` folds scaling and dequantization into an undocumented datapath
- Format semantics are vendor-defined — NVFP4 cannot fall back on IEEE-754
- "Reproduce this NVFP4 RL run on another GPU" has no reference to compare against

---

1 Characterise the numerics of low-bit matrix instructions

2 A layered consistency budget: which layers must be bitwise, which only need top-K not to flip

3 The minimum contract for heterogeneous RL — and its cost curve

Summary

Thinking Machines	Kernel	batch-invariant kernels; all traffic pays (26 s → 55 s)
LLM-42	Engine	decode–verify–rollback; pay per deterministic request
vLLM / DeepSeek-V4	Across engines	training calls the inference kernels; diff = 0 at 91.3%
ReproFuse / Verde / Modal	Across hardware	open: no reference semantics for low-bit

→ From an engine-wide switch to something you buy by scope and by traffic

Thanks

Questions and discussion